

CS112: Friday 18 September, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

Wednesday we read files and threw exceptions: streams that fail silently, the `while (!fin.eof())` trap, and `throw` unwinding outward until something catches it.

Today: a grid is not a grid. A two-dimensional array is one contiguous run of memory with an agreement about where the rows begin, which is week 0's pointer arithmetic wearing a second set of brackets.

TIME	BEAT
12:20	1. Declaring and walking a grid
12:38	2. What it really looks like in memory
12:55	3. Passing one to a function
13:08	4. Printing it so a human can read it

Beat 1. Declaring and walking a grid (18 min)

```
char grid[4][6];

for (int r = 0; r < 4; ++r)
    for (int c = 0; c < 6; ++c)
        grid[r][c] = '.';
```

The outer loop picks a row, the inner one walks it. Which index is which is a convention you adopt and then must not break halfway through a program.

Beat 2. What it really looks like in memory (17 min)

```
cout << &grid[0][0] << endl;
cout << &grid[0][5] << endl;
cout << &grid[1][0] << endl;    // one past the end of row 0
```

There are no rows. There are 24 characters in a row, and `grid[1][0]` is the seventh of them. The compiler works it out:

$$\text{grid}[r][c] \equiv *(\text{start} + r \cdot 6 + c)$$

Talk to your neighbor. Where does the 6 come from, and why can the compiler not get it from the first index?

Beat 3. Passing one to a function (13 min)

```
void show(char g[][6], int rows);    // the 6 is required
```

That is the formula, made concrete. A function handed the address of the first element can find `[r][c]` only if it is told how long a row is. The number of rows it can be told at run time; the width has to be in the type.

Beat 4. Printing it so a human can read it (10 min)

`setw()` and the stream manipulators from week 0, put to work lining up columns. A grid you cannot read is a grid you cannot debug.